

兆鸣嵌入式

C语言·一个LED讲透面向对象

EP06 封装：struct与me指针

三家世界500强 · 11年一线嵌入式实战

公众号搜索「兆鸣嵌入式」 回复「交流」进群

核心概念

封装的本质：把描述同一个事物的数据打包在一起（struct），再通过me指针让同一份函数服务不同的数据。

封装不是藏代码——是让同一份逻辑服务不同的数据

设计思路

问题：三个LED写三份重复代码

传统写法：red_on()、green_on()、blue_on()——三个LED，三份几乎一模一样的代码。如果要改亮度逻辑，得改三个地方，漏改一个就出bug。

解决：struct打包数据 + me指针统一操作

用Led_t结构体把一颗LED的所有数据（pin、brightness、is_on）打包在一起，就像一张"身份证"。每个函数的第一个参数是Led_t

*me——"我操作的是哪颗LED"。同一份led_on(me)代码，传不同的me指针，就操作不同的LED。

C++里写 led.on()，编译器偷偷传了一个this指针；我们在C里手动传me，效果完全一样。

视频讲义

三份代码，做的事情一模一样

区别只有引脚号 —— 改一个地方要改三遍

红灯

逻辑相同

```
void red_led_on(void)
{
    HAL_GPIO_WritePin(
        GPIOI,
        GPIO_PIN_15,
        GPIO_PIN_SET);
}
```

绿灯

逻辑相同

```
void green_led_on(void)
{
    HAL_GPIO_WritePin(
        GPIOI,
        GPIO_PIN_14,
        GPIO_PIN_SET);
}
```

蓝灯

逻辑相同

```
void blue_led_on(void)
{
    HAL_GPIO_WritePin(
        GPIOI,
        GPIO_PIN_13,
        GPIO_PIN_SET);
}
```

只有引脚号不同 → 能不能代码只写一份？

兆鸣嵌入式

三个LED写了三份重复代码——问题的起点

给每个LED发一张"挂号单"

C语言里这种单子叫 struct —— 每张单子格式一样，内容不同

就像医院的挂号单

红灯的单子

引脚号 15
点亮电平 高电平

绿灯的单子

引脚号 14
点亮电平 高电平

蓝灯的单子

引脚号 13
点亮电平 高电平

写成代码



C语言的"挂号单" = struct

```
typedef struct
{
    int32_t pin_num; /* 引脚号 */
    bool light_level; /* 点亮电平 */
} led_t;
```

红灯

引脚 15

绿灯

引脚 14

蓝灯

引脚 13

数据写在单子上，函数只写一份

兆鸣嵌入式

struct就像挂号单，把数据打包在一起

递不同的单子，操作不同的LED

函数只写一份 —— me 告诉它"现在操作谁"

函数只有一份，通过 me 区分

```
/* 点亮函数: me 告诉它操作哪个LED */
void led_on(led_t *me)
{
    platform_pin_write(
        me->pin_num,
        me->light_level);
}

/* 三张单子，三个独立的LED */
led_t led_red   = { .pin_num = 15 };
led_t led_green = { .pin_num = 14 };
led_t led_blue  = { .pin_num = 13 };

/* 递不同的单子 → 操作不同的LED */
led_on(&led_red);    /* 红灯亮 */
led_off(&led_green); /* 绿灯灭 */
led_on(&led_blue);   /* 蓝灯亮 */
```

当前状态

R

红灯

亮

G

绿灯

灭

B

蓝灯

亮

一份代码，想要多少个LED就多少个 —— 再发一张单子就行

兆鸣嵌入式

me指针递单子——同一份函数服务不同数据

C语言 vs C++：底层原理一模一样

C++ 只是让编译器帮你干了这件事

C语言 你手动做

```
struct + me 指针

/* 数据: 你自己定义struct */
typedef struct {
    int32_t pin_num;
    bool light_level;
} led_t;

/* 函数: 你自己传me */
void led_on(led_t *me)
{
    pin_write(me->pin_num,
              me->light_level);
}

/* 使用 */
led_t red = { 15, 1 };
led_on(&red);
```

=
底层相同

C++ 编译器帮你做

```
class + 隐藏的 this

/* 数据: 编译器帮你包进class */
class Led {
    int32_t pin_num;
    bool light_level;

/* 函数: 编译器偷偷传this */
    void on()
    {
        pin_write(this->pin_num,
                  this->light_level);
    }
};

/* 使用 */
Led red(15, 1);
red.on();
```

恭喜你，你已经理解了面向对象最核心的概念

你不是背会的 —— 是你自己推出来的

兆鸣嵌入式

C vs C++ 对照: struct+me = class+this

代码结构

文件	说明
led.h	LED模块公共接口——struct定义 + 函数声明
led.c	LED模块实现——每个函数第一个参数都是 Led_t *me
main.c	演示程序——三颗LED，一份代码
Makefile	编译脚本

关键代码解读

1. Led_t结构体——把数据打包在一起

led.h

```
typedef struct {
    uint8_t pin;        /* GPIO引脚号 */
    uint8_t brightness; /* 亮度, 0~100 */
    bool    is_on;      /* 开关状态 */
} Led_t;
```

一张"身份证": pin是这颗LED接在哪个引脚, brightness是当前亮度, is_on是亮着还是灭着。三个数据打包在一起, 走到哪带到哪。

2. led_on函数——me指针决定"操作谁"

led.c

```
int led_on(Led_t *me)
{
    if (me == NULL) {
        return -1;
    }
    me->is_on = true;
    platform_gpio_write(me->pin, true);
    printf(" [LED] Pin%d ON\n", me->pin);
    return 0;
}
```

函数只有一份, 但me指针指向谁, 就操作谁。传&red_led就亮红灯, 传&green_led就亮绿灯。

3. main.c——三颗LED，一份代码

main.c

```
Led_t red_led;    /* 红灯 */
Led_t green_led;  /* 绿灯 */
Led_t blue_led;   /* 蓝灯 */

led_init(&red_led, 13);
led_init(&green_led, 14);
led_init(&blue_led, 15);

led_on(&red_led);  /* 亮红灯 */
led_on(&green_led); /* 亮绿灯 */
led_on(&blue_led); /* 亮蓝灯 */
```

三个Led_t变量就是三颗LED的全部数据，led_on()只有一份，me指针决定操作哪颗——这就是封装。

C vs C++ 对照

C 写法	C++ 写法
Led_t (struct)	class Led
Led_t *me	this指针（编译器隐式传递）
led_on(Led_t *me)	Led::on()
led_init(&red_led, 13)	Led red_led(13)
led_deinit(&red_led)	~Led()（自动调用）

编译运行

```
cd code/EP06_封装
make
./demo
```

获取更多

公众号搜索「兆鸣嵌入式」

回复「交流」加群，与一线工程师切磋

抖音 / B站 / 视频号

搜索「兆鸣嵌入式」